

WarrantyPatternMiner

Complete Build Roadmap - Hackathon Edition

Product: WarrantyPatternMiner **Tagline:** Catching Defect Trends Before They Become Recalls **Purpose:** Build a working AI early-warning system that mines warranty/complaint narratives, finds hidden defect clusters across inconsistent failure codes, detects statistically unusual growth, and gives reliability engineers evidence-linked alerts for human verification.

1. What You Are Building

WarrantyPatternMiner ingests warranty/field-complaint records containing:

- Claim ID
- Date
- Product/model
- Plant/service location
- Structured failure code
- Free-text technician/complaint narrative

The central product thesis is that the structured code is often a weak or generic label, while the free-text narrative contains the diagnostic signal. The system therefore extracts the underlying failure signature from narrative text, compares it with the assigned code, embeds the narrative semantically, discovers cross-code clusters, tests whether cluster volume is changing unusually, and presents the evidence to a human reliability engineer.

The core loop is:

INPUT -> EXTRACT -> COMPARE -> EMBED -> CLUSTER -> DETECT
TREND -> ALERT -> VERIFY -> LEARN

The system must not autonomously make recall decisions or external regulatory filings. A human is the final decision gate.

2. Hackathon Objective

The best hackathon version is NOT an enterprise platform. It is a focused, polished proof that demonstrates one powerful capability:

“The dashboard says there is no defect. We read the narratives and discover that 15 claims across 5 codes are actually the same emerging problem.”

Your MVP should prove four things:

1. Narrative text contains signal hidden from structured-code analysis.
 2. AI can semantically group differently worded claims.
 3. A statistical layer can distinguish emerging patterns from ordinary noise.
 4. Every alert can be traced to source claims and reviewed by a human.
-

3. Recommended Scope

Must Have

- CSV/JSON upload or fixed demo dataset
- Narrative extraction into structured failure signatures
- Embeddings
- Unsupervised clustering
- Cross-code cluster detection
- Time-based trend analysis
- Emerging-pattern scoring
- Evidence-linked cluster drill-down
- Confirm / dismiss workflow
- Persistent storage of analyst decisions
- Clean web dashboard

Strongly Recommended

- Baseline comparison against structured-code-only analysis
- Mis-coded claim detection
- Defect fingerprint library
- Natural-language alert summary
- Claim-level semantic similarity explanation
- Demo dataset with a guaranteed hidden cluster

Do NOT Build for the Hackathon

- Kafka/event-streaming infrastructure
 - Real OEM warranty-system integrations
 - Automated recall filing
 - Automated regulatory communication
 - Mobile application
 - Complex multi-tenant SaaS/authentication
 - Production-scale model training
 - Generic text-mining platform
-

4. User Personas

Primary - Reliability / Quality Engineer

Responsible for field failures, emerging defect investigation, reliability trends, and engineering escalation.

Needs: - Early signals - Evidence - Trend history - Affected models/plants - Low false-positive noise - Ability to verify and annotate AI findings

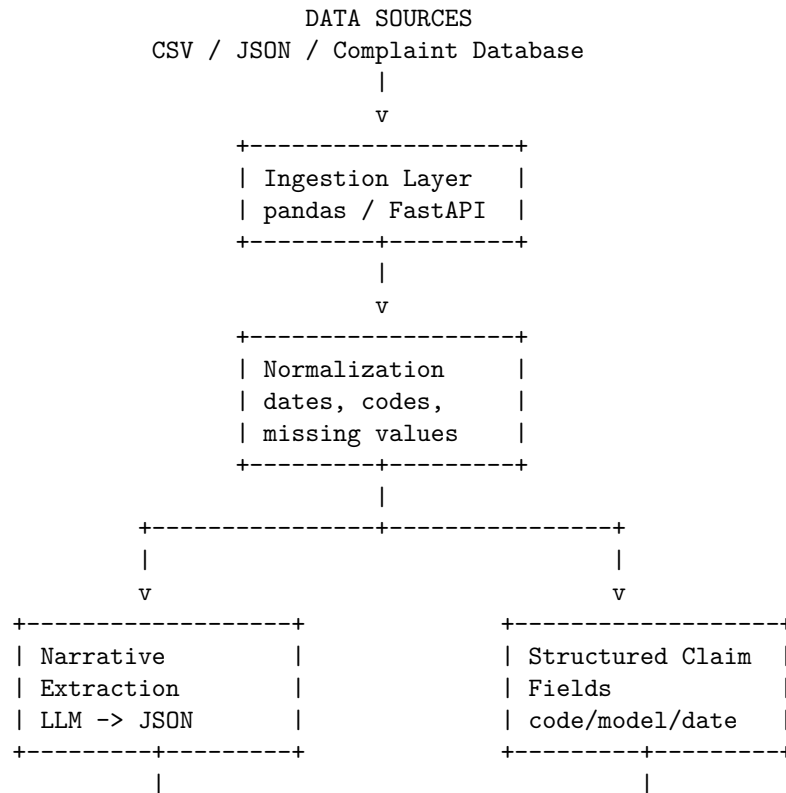
Secondary - Warranty Operations Manager

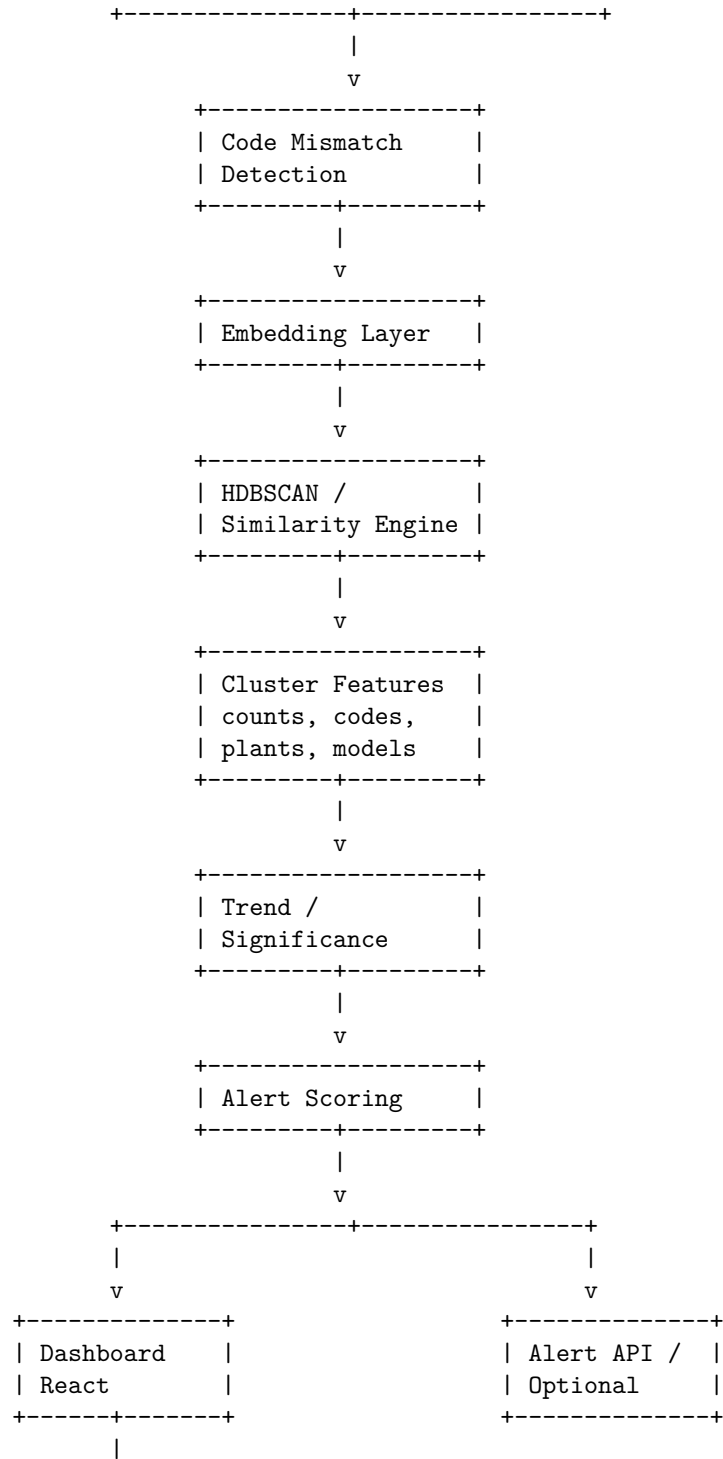
Sees claim volume and cost but needs help explaining why patterns are changing.

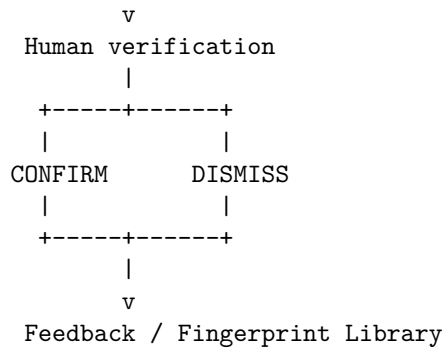
Executive Stakeholder - VP Quality / Reliability Leader

Cares about recall risk, warranty exposure, customer impact, and response time.

5. End-to-End Architecture







6. Suggested Tech Stack

Backend

- Python 3.11+
- FastAPI
- Pydantic
- pandas
- NumPy
- scikit-learn
- hdbscan
- SciPy
- SQLAlchemy

AI

- LLM API for structured extraction and concise explanations
- Embedding model/API for semantic representations

Keep model calls narrow and structured. The LLM should not control the whole pipeline.

Database

Recommended: - PostgreSQL - pgvector if convenient

For a very fast MVP, SQLite can work locally, but PostgreSQL gives a cleaner path to production.

Frontend

Recommended: - React - TypeScript - Vite - Tailwind CSS - Recharts or another lightweight charting library

For an ultra-fast prototype, Streamlit is acceptable, but React will generally provide a stronger hackathon-quality product experience.

Deployment

Simple options: - Frontend: Vercel - Backend: Render / Railway / Fly.io - Database: managed PostgreSQL

A local demo is acceptable if the judging environment makes hosted services risky.

7. Repository Structure

```
warranty-pattern-miner/
|
+-- apps/
|   +-- api/
|       |   +-- main.py
|       |   +-- routes/
|       |       |   +-- claims.py
|       |       |   +-- clusters.py
|       |       |   +-- alerts.py
|       |       |   +-- feedback.py
|       |   +-- services/
|       |       |   +-- extraction.py
|       |       |   +-- embeddings.py
|       |       |   +-- clustering.py
|       |       |   +-- trends.py
|       |       |   +-- scoring.py
|       |   +-- models/
|       |   +-- schemas/
|       |   +-- db/
|       |
|   +-- web/
|       +-- src/
|           +-- pages/
|           +-- components/
|           +-- hooks/
|           +-- api/
|           +-- types/
|
+-- data/
|   +-- raw/
|   +-- processed/
```

```

|   +-- demo/
|
+-- pipelines/
|   +-- ingest.py
|   +-- extract.py
|   +-- embed.py
|   +-- cluster.py
|   +-- trend.py
|   +-- run_pipeline.py
|
+-- tests/
|   +-- unit/
|   +-- integration/
|   +-- fixtures/
|
+-- scripts/
|   +-- generate_demo_data.py
|   +-- seed_database.py
|   +-- run_demo.py
|
+-- docs/
|   +-- architecture.md
|   +-- demo-script.md
|   +-- evaluation.md
|
+-- .env.example
+-- README.md
+-- docker-compose.yml

```

8. Data Model

claims

```

claims (
  id UUID PRIMARY KEY,
  external_claim_id TEXT UNIQUE NOT NULL,
  claim_date DATE NOT NULL,
  product_model TEXT,
  plant TEXT,
  failure_code TEXT,
  narrative TEXT NOT NULL,
  source TEXT,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
)

```

)

failure__signatures

```
failure_signatures (  
  id UUID PRIMARY KEY,  
  claim_id UUID REFERENCES claims(id),  
  component TEXT,  
  symptom TEXT,  
  condition TEXT,  
  severity TEXT,  
  inferred_failure TEXT,  
  contributing_factors JSONB,  
  extraction_confidence FLOAT,  
  created_at TIMESTAMP  
)
```

embeddings

```
embeddings (  
  claim_id UUID REFERENCES claims(id),  
  vector VECTOR,  
  model_name TEXT,  
  created_at TIMESTAMP  
)
```

clusters

```
clusters (  
  id UUID PRIMARY KEY,  
  run_id UUID,  
  label TEXT,  
  description TEXT,  
  claim_count INT,  
  growth_rate FLOAT,  
  significance_score FLOAT,  
  anomaly_score FLOAT,  
  created_at TIMESTAMP  
)
```

cluster__claims

```
cluster_claims (  
  cluster_id UUID REFERENCES clusters(id),  
  claim_id UUID REFERENCES claims(id),  
  similarity_score FLOAT,
```



```

    PRIMARY KEY (cluster_id, claim_id)
)

```

feedback

```

feedback (
  id UUID PRIMARY KEY,
  cluster_id UUID REFERENCES clusters(id),
  decision TEXT CHECK (decision IN ('confirmed','edited','dismissed')),
  rationale TEXT,
  reviewer TEXT,
  created_at TIMESTAMP
)

```

defect__fingerprints

```

defect_fingerprints (
  id UUID PRIMARY KEY,
  name TEXT,
  description TEXT,
  component TEXT,
  symptoms JSONB,
  examples JSONB,
  confirmed_count INT,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
)

```

9. Data Contract

Every ingested record should conform to:

```

{
  "claim_id": "C-10021",
  "date": "2026-05-17",
  "product_model": "Model X",
  "plant": "Plant A",
  "failure_code": "OTHER",
  "narrative": "Customer reports clunking noise from front left when driving over rough road"
}

```

Validation rules:

- claim_id required
- date required

- narrative required
- failure_code may be missing but should be preserved as null
- trim whitespace
- normalize dates
- remove impossible dates
- preserve raw narrative exactly in a raw field

Never overwrite original source text with an AI-generated version.

10. Dataset Strategy

Best Hackathon Dataset

Use a hybrid:

1. Real public complaint/warranty-like data for credibility.
2. Controlled synthetic injection for a guaranteed hidden-cluster reveal.

The synthetic injection should be intentionally messy.

Do NOT create 20 nearly identical sentences.

Instead create semantically related narratives:

- “front end knocks over speed bumps”
- “metallic tap near driver’s wheel on potholes”
- “intermittent clunk from left suspension”
- “steering-side front assembly produces knocking noise”
- “no electrical fault found, customer hears tapping from front corner”

Assign these to different generic failure codes.

Example distribution:

OTHER	5
RIDE QUALITY	4
SUSPENSION	3
ELECTRICAL-NFF	3
STEERING	2

Underlying cluster:

FRONT-LEFT SUSPENSION NOISE
17 claims

11. Dataset Generation Checklist

Generate at least:

- 500-2,000 realistic background claims
- 15-25 hidden-cluster claims
- 5-10 obvious/non-emerging clusters
- A mixture of dates across several months
- Multiple products/models
- Multiple plants/locations
- Multiple structured failure codes
- Some deliberately mis-coded claims
- Some narratives with low-information language
- Some duplicate-like claims for similarity tests

Keep a ground-truth file containing the intended hidden clusters. This is essential for evaluation.

Example:

```
{
  "ground_truth_clusters": [
    {
      "name": "front_left_suspension_noise",
      "claim_ids": ["C9001", "C9002", "C9003"]
    }
  ]
}
```

12. Phase 0 - Project Setup

Goal

Get the environment reproducible.

Tasks

- Create Git repository
- Create Python virtual environment
- Create frontend application
- Add environment variables
- Add linting/formatting
- Add README
- Add sample dataset
- Add Docker Compose if using PostgreSQL

Definition of Done

Backend starts
Frontend starts
Database connects
Sample claims can be loaded
README explains setup

13. Phase 1 - Ingestion Pipeline

Goal

Get raw claims into a normalized table.

Tasks

1. CSV loader
2. JSON loader
3. Schema validation
4. Missing-value handling
5. Date normalization
6. Source tracking
7. Database insertion
8. Duplicate detection

API

POST /claims/upload
GET /claims
GET /claims/{claim_id}

Definition of Done

Upload 1,000 claims and retrieve them through API and UI.

14. Phase 2 - AI Narrative Extraction

Goal

Convert messy narrative into deterministic structured information.

Output Schema

```
{
  "component": "front-left suspension",
  "symptom": "clunking noise",
  "condition": "rough roads",
  "severity": "moderate",
  "inferred_failure": "excessive play",
  "contributing_factors": [],
  "confidence": 0.91
}
```

Prompt Requirements

- Return JSON only
- Do not invent facts
- Use null when unsupported
- Distinguish observed symptom from inferred cause
- Keep confidence separate from certainty

Engineering Rules

- Cache extraction results
- Retry transient failures
- Log model/version
- Validate JSON with Pydantic
- Store raw model output for debugging

Definition of Done

At least 95% of test narratives produce schema-valid outputs.

15. Phase 3 - Structured-Code Mismatch Detection

Goal

Determine whether the assigned failure code agrees with the narrative-derived signature.

Example:

Assigned code: ELECTRICAL-NFF

AI signature: front-left suspension / mechanical clunk

Result: MISMATCH
Confidence: 0.94

Approach

Start simple.

Create a normalized description for each structured code:

SUSPENSION -> suspension/ride/strut/control arm/etc.

ELECTRICAL-NFF -> no electrical fault / electrical diagnosis

BRAKES -> braking / rotor / pad / brake noise

Compare the inferred signature to the code meaning using:

- semantic similarity
- lightweight classifier
- or LLM judgment constrained to a strict schema

Output

```
{  
  "is_mismatch": true,  
  "expected_category": "suspension",  
  "assigned_category": "electrical",  
  "confidence": 0.94  
}
```

16. Phase 4 - Embedding Pipeline

Goal

Represent narrative meaning mathematically.

For each claim:

narrative -> embedding -> vector

Recommended embedding text

Do not embed only the raw sentence blindly.

Create a normalized semantic representation such as:

Component: front-left suspension

Symptom: clunking/knocking

Condition: rough roads

Failure: excessive play

Raw narrative: ...

This gives the clustering system richer information while retaining evidence.

Tasks

- Batch embedding
- Cache embeddings
- Handle failures/retries
- Store model name/version
- Normalize if required

Definition of Done

Every valid claim has an embedding and can be queried for nearest semantic neighbors.

17. Phase 5 - Clustering

Goal

Discover groups without knowing cluster count in advance.

Initial Method

HDBSCAN.

Inputs:

`claim embeddings`

Outputs:

`cluster label`

`cluster membership probability`

`noise/outlier label`

Important

Do NOT cluster only within failure-code categories.

The whole point is:

ALL CODES -> COMMON SEMANTIC SPACE -> CLUSTERS

Cluster quality checks

Reject or down-rank clusters that:

- Have too few claims
- Have very low internal similarity

- Are dominated by generic language
- Show no meaningful common features

Definition of Done

The injected hidden cluster is recovered with high precision/recall on the demo dataset.

18. Phase 6 - Cluster Labeling

Raw cluster IDs such as:

`cluster_7`

are useless to users.

Generate a human-readable label and summary.

Example:

`Front-left suspension knocking`

Claims in cluster repeatedly describe knocking, clunking or metallic tapping around the driver's-side front suspension, especially over uneven surfaces.

Use representative claims and extracted signatures as the LLM context.

Do not send thousands of claims into the LLM.

Select: - representative examples - common extracted fields - top terms - statistics

19. Phase 7 - Trend Detection

Goal

Determine which semantic clusters are genuinely emerging.

For each cluster, calculate:

`count by week/month`

`baseline volume`

`recent volume`

`growth rate`

`acceleration`

MVP Algorithm

A simple rolling baseline is sufficient.

Example:

```
baseline_mean = average(previous N periods)
baseline_std = standard deviation(previous N periods)
recent = current period count
z = (recent - baseline_mean) / baseline_std
```

Add a minimum-volume guard so tiny clusters do not create noise.

Optional CUSUM

For a stronger statistical story, implement CUSUM as an additional detector.

Emerging score

Create a combined score from:

- recent growth
- statistical surprise
- cluster size
- semantic coherence
- cross-code spread
- persistence over time

Keep the score explainable.

20. Example Alert Scoring

One simple formulation:

```
alert_score =
    0.30 * normalized_growth +
    0.25 * significance +
    0.15 * cluster_size_score +
    0.15 * cross_code_score +
    0.15 * coherence_score
```

Then classify:

80-100	CRITICAL
60-79	HIGH
40-59	WATCH
0-39	NORMAL

These thresholds are demo defaults, not validated regulatory thresholds.

21. Phase 8 - Evidence Engine

This is one of the most important parts of the product.

Every alert must answer:

“Why did you tell me this?”

For every cluster provide:

- Cluster summary
- Claim count
- Trend chart
- Failure-code distribution
- Model distribution
- Plant distribution
- Date distribution
- Representative claims
- Similarity evidence
- Mismatch statistics

Example:

Why this pattern was flagged

23 claims
5 different failure codes
3 plants
+312% vs baseline

Representative evidence:

C-1827: "clunking noise front left..."

C-1932: "knocking near driver-side wheel..."

C-2071: "metallic tapping from front corner..."

22. Phase 9 - Dashboard

Build the UI around investigation, not data dumping.

Screen A - Command Center

Top metrics:

Claims analyzed	12,482
Emerging patterns	7

High-risk patterns	2
Mis-coded claims	431

Main panel:

EMERGING PATTERN

Front-left suspension knocking

+312% vs baseline

23 claims | 5 codes | 3 plants

Screen B - Pattern Investigation

Sections:

1. Summary
2. Trend chart
3. Code distribution
4. Product/model distribution
5. Plant distribution
6. Evidence claims
7. AI rationale
8. Decision controls

Screen C - Claim Detail

Display original narrative prominently.

Never bury the source evidence behind multiple interactions.

Screen D - Feedback

[Confirm pattern]

[Edit interpretation]

[Dismiss]

Rationale: _____

23. Phase 10 - Feedback Loop

Goal

Make analyst decisions part of the product's memory.

When confirmed:

- Save decision
- Save rationale

- Add representative examples to fingerprint library
- Increase confidence in matching patterns

When dismissed:

- Save decision
- Save reason
- Mark the cluster as a known false-positive pattern
- Use it in evaluation and future filtering

MVP Learning Mechanism

Do not attempt online model retraining in the hackathon.

Instead, make the feedback loop operational through a persistent knowledge layer:

```
Confirmed cluster
    |
    v
Defect fingerprint
    |
    v
Future similarity comparison
```

This is simpler, explainable, and demo-friendly.

24. Phase 11 - Defect Fingerprint Library

Each confirmed defect can become:

```
{
  "name": "Front-left suspension knocking",
  "component": "front-left suspension",
  "symptoms": [
    "clunk",
    "knock",
    "metallic tapping"
  ],
  "conditions": [
    "rough road",
    "pothole",
    "low speed"
  ],
  "example_claims": ["C-1827", "C-1932"]
}
```

Then future claims can be compared with known defect fingerprints.

This gives the product a memory layer rather than making every analysis stateless.

25. Phase 12 - Baseline Comparison View

This is a major demo enhancement.

Show two views of the same data.

Traditional

Code-level monitoring

OTHER	5
RIDE QUALITY	4
SUSPENSION	3
ELECTRICAL-NFF	3
STEERING	2

No alert.

WarrantyPatternMiner

Semantic cluster

Front-left suspension defect
17 claims
5 different codes
+287% vs baseline

ALERT

This single comparison explains the product better than paragraphs of text.

26. Evaluation Framework

You need quantitative proof that the system works.

Extraction Metrics

- JSON validity rate

- Component extraction accuracy
- Symptom extraction accuracy
- Mismatch classification accuracy

Clustering Metrics

With ground truth:

- Precision
- Recall
- F1
- Adjusted Rand Index where appropriate
- Cluster purity

Detection Metrics

- True emerging cluster detected
- False alert rate
- Detection lead time

Most compelling metric

Detection lead time:

Structured threshold reached: Day 60

WarrantyPatternMiner signal: Day 42

Lead time: 18 days

This turns the story from “our AI found a cluster” into “our AI found it earlier.”

27. Golden Test Cases

Create a fixed suite before polishing the UI.

Test 1 - Same words, same defect

Should cluster strongly.

Test 2 - Different words, same defect

Should cluster semantically.

Test 3 - Same component, unrelated failures

Should not collapse everything together.

Test 4 - Different codes, same defect

Must cluster across code boundaries.

Test 5 - Increasing cluster

Must alert.

Test 6 - Stable cluster

Must remain non-alerting.

Test 7 - One-off outlier

Should not become a high-confidence emerging pattern.

Test 8 - False semantic similarity

Should be reviewable and dismissible.

28. Demo Data Design

Build one canonical hidden defect.

Example timeline:

Month	Claims in hidden cluster
Jan	1
Feb	1
Mar	2
Apr	2
May	4
Jun	8
Jul	17

At the same time, distribute codes:

OTHER	6
RIDE QUALITY	4
SUSPENSION	3
NFF	2
STEERING	2

Traditional analysis sees fragmented categories.

The semantic layer sees one growing problem.

29. Recommended Build Order

Do not build frontend-first.

Build in this exact order:

1. Dataset
2. Ingestion
3. Extraction
4. Embeddings
5. Clustering
6. Trend detection
7. Evidence API
8. Dashboard
9. Feedback loop
10. Demo polish

This prevents you from building a beautiful UI around an unreliable detection pipeline.

30. 24-36 Hour Hackathon Execution Plan

Hours 0-2: Foundation

- Repository
- Environment
- Dataset
- Database
- Basic backend

Deliverable: claims are loaded and queryable.

Hours 2-5: Extraction

- LLM schema
- Extraction service
- Validation
- Caching

Deliverable: every claim has a failure signature.

Hours 5-8: Embeddings

- embedding service
- vector storage
- nearest-neighbor sanity tests

Deliverable: similar claims retrieve each other.

Hours 8-12: Clustering

- HDBSCAN
- cluster stats
- representative claims
- cross-code counts

Deliverable: hidden cluster appears.

Hours 12-15: Trend engine

- time aggregation
- baseline
- significance
- alert score

Deliverable: emerging cluster gets flagged.

Hours 15-20: Dashboard

- command center
- cluster details
- evidence
- charts

Deliverable: usable investigation workflow.

Hours 20-23: Human verification

- confirm
- dismiss
- rationale
- fingerprint creation

Deliverable: closed loop.

Hours 23-27: Comparison demo

- structured-only baseline
- semantic detector
- reveal view

Deliverable: judge-ready value demonstration.

Hours 27-30: Testing

- golden cases

- fix false positives
- stabilize API
- seed deterministic dataset

Hours 30-36: Polish + Demo

- visual polish
 - loading states
 - clear copy
 - screenshots
 - 3-minute rehearsal
 - backup local demo
 - video recording
-

31. API Design

Claims

```
POST /claims/upload
GET /claims
GET /claims/{id}
```

Analysis

```
POST /analysis/run
GET /analysis/runs/{run_id}
```

Clusters

```
GET /clusters
GET /clusters/{id}
```

Alerts

```
GET /alerts
GET /alerts/{id}
```

Feedback

```
POST /clusters/{id}/feedback
```

Fingerprints

```
GET /fingerprints
POST /fingerprints
```

32. Analysis Run Object

Every pipeline execution should get a run ID.

```
{
  "run_id": "RUN-2026-08-30-01",
  "dataset_version": "demo-v3",
  "embedding_model": "...",
  "extraction_model": "...",
  "cluster_algorithm": "hdbscan",
  "thresholds": {
    "min_cluster_size": 4,
    "alert_threshold": 60
  },
  "created_at": "..."
}
```

This is extremely useful for reproducibility.

33. Reliability Requirements

The demo must never depend on an unpredictable model response at runtime.

Recommended strategy:

- Precompute embeddings for the canonical demo dataset.
- Precompute extraction results where practical.
- Cache all model outputs.
- Allow the UI to rerun the analysis deterministically.
- Keep a local copy of the demo data.
- Keep screenshots/video as fallback.

The live demo can still show the pipeline running, but the underlying outputs should be reproducible.

34. Prompt Design Principles

The LLM should be used where language reasoning matters.

Good use:

Narrative -> structured failure signature

Good use:

Cluster evidence -> concise human explanation

Bad use:

Raw dataset -> ask an LLM "find defects"

Bad use:

LLM decides statistical significance

Statistical decisions should remain deterministic and inspectable.

35. Explainability Requirements

Every AI output should have an explanation path.

For a cluster:

WHY GROUPED?

Semantic similarity

- + shared component
- + shared symptom
- + similar operating condition

WHY ALERTED?

Recent volume > historical baseline

- + statistically unusual growth
- + multiple structured codes involved

Do not expose meaningless internal embedding dimensions to users.

36. Security / Privacy for Demo

Because warranty claims may contain sensitive business information:

- Avoid hardcoding production credentials.
 - Do not commit secrets.
 - Store API keys in environment variables.
 - Avoid uploading private customer identifiers.
 - Use synthetic or public data for the hackathon.
 - Strip unnecessary personal information from demo claims.
-

37. Production Evolution Path

The hackathon MVP can later evolve into:

Stage 1

Batch CSV analysis.

Stage 2

API/webhook ingestion.

Stage 3

Continuous event-driven detection.

Stage 4

Integration with warranty systems.

Stage 5

Organization-specific defect fingerprint memory.

Stage 6

Cross-product failure intelligence.

Stage 7

Investigation workflow and 8D assistance.

The product should remain human-in-the-loop around consequential decisions.

38. Post-Hackathon Product Roadmap

V1 - Narrative Intelligence

- Better extraction
- Better clustering
- Better evidence
- More product categories

V2 - Continuous Monitoring

- Streaming ingestion
- Scheduled analysis
- Alert routing
- Slack/email integrations

V3 - Investigation System

- 8D generation
- root-cause evidence collection
- engineering task creation
- affected population analysis

V4 - Defect Intelligence Platform

- known defect fingerprints
 - cross-model knowledge
 - supplier correlation
 - production-batch correlation
 - component genealogy
-

39. Supplier / Manufacturing Expansion

A future differentiator could be connecting field defects to upstream manufacturing information:

Field claims

|

v

Defect cluster

|

+--> model

+--> plant

+--> build date

+--> supplier

+--> component batch

|

v

Potential root-cause population

This is not necessary for the hackathon but gives the concept substantial expansion potential.

40. UI Design Principles

The UI should feel like an investigation console, not a generic SaaS dashboard.

Principles:

- Spacious
- Evidence-first
- Minimal visual clutter
- Strong hierarchy
- One primary action per screen
- Trends before tables
- Source claim always one click away
- Severity should be obvious without relying solely on color

The most important visual object is the emerging-pattern card.

41. The Ideal Alert Card

```
+-----+
| EMERGING DEFECT                                     |
|                                                     |
| Front-left suspension knocking                       |
|                                                     |
| +312% vs baseline                                   |
| 23 claims    5 codes    3 plants                     |
|                                                     |
| First detected: 18 days ago                         |
|                                                     |
| "Multiple narratives describe similar                 |
| knocking/clunking around the front-left             |
| suspension despite different failure codes."         |
|                                                     |
| [ INVESTIGATE ]                                     |
+-----+
```

This should be the hero component in the demo.

42. The 3-Minute Demo Script

0:00-0:30 - Problem

Say:

“Every warranty claim has a structured failure code and a free-text narrative. Manufacturers analyze the code because it is easy to aggregate. But the diagnostic signal often lives in the narrative.”

Show one claim where the code is generic but the narrative is specific.

0:30-1:15 - Run Analysis

Load the claims.

Show the system extracting signatures and forming semantic clusters.

1:15-1:50 - Reveal

Open the hidden cluster.

Show:

- 17 claims
- 5 different failure codes
- 3 plants
- multiple wording variants

1:50-2:25 - Trend

Show the cluster timeline.

Explain that structured categories individually remain below alert threshold, while the combined semantic cluster is growing abnormally.

2:25-2:45 - Evidence

Open source claims.

Highlight the evidence.

2:45-3:00 - Human Gate

Click Confirm.

Show:

Pattern confirmed.

Added to defect fingerprint library.

Close with:

“We are not automating recall decisions. We are making the evidence visible early enough for engineers to act.”

43. Demo Failure-Proofing

Prepare three modes.

Mode 1 - Live

Run pipeline live with cached AI outputs.

Mode 2 - Seeded

Load precomputed results immediately.

Mode 3 - Video

Recorded backup.

The judge should never see a blank dashboard because an external API timed out.

44. Hackathon Submission Assets

You need:

README

Include:

- problem
- solution
- architecture
- setup
- dataset
- AI components
- evaluation
- screenshots
- demo instructions

Demo Video

Maximum 5 minutes.

Keep the narrative focused on:

Problem -> Hidden pattern -> Detection -> Evidence -> Outcome

Presentation

Recommended 8-10 slides:

1. Problem
 2. Why current systems miss it
 3. Product
 4. Hidden cluster example
 5. Workflow
 6. Technical architecture
 7. Evaluation/results
 8. UX/demo
 9. Business potential
 10. Roadmap
-

45. Judge Questions You Should Expect

“Why not just search keywords?”

Answer:

Keywords fail when different technicians use different language. Semantic representations can connect conceptually related narratives even when vocabulary differs.

“Why not just use an LLM?”

Answer:

The LLM extracts semantic structure; embeddings perform similarity; clustering discovers groups; statistical analysis determines whether growth is abnormal. Each component has a specific responsibility.

“How do you prevent false positives?”

Answer:

Evidence-linked human verification is mandatory, with feedback and an organization-specific defect fingerprint layer.

“Does this make recall decisions?”

Answer:

No. It surfaces evidence and emerging patterns. Engineers remain responsible for consequential decisions.

“What is your moat?”

Answer:

Over time, confirmed patterns, false-positive history, domain vocabulary, and defect fingerprints become organization-specific operational knowledge.

“Why now?”

Answer:

Modern language models and embedding systems make large-scale semantic interpretation of messy field narratives practical without requiring teams to manually read every claim.

46. What Success Looks Like

By the final demo, the following should all work:

- [] Upload dataset
- [] Normalize claims
- [] Extract failure signatures
- [] Detect code mismatches
- [] Embed claims
- [] Discover semantic clusters
- [] Calculate cluster trends
- [] Flag emerging patterns
- [] Open supporting claims
- [] Compare against structured-only analysis
- [] Confirm/dismiss pattern
- [] Create fingerprint
- [] Re-run analysis successfully

47. Final Technical Checklist

Before submission:

Data

- ☐ Dataset deterministic
- ☐ Ground truth available
- ☐ No unnecessary PII

AI

- ☐ Extraction schema validated
- ☐ Embeddings cached
- ☐ Clustering reproducible
- ☐ LLM prompts versioned

Analytics

- ☐ Trend calculations tested
- ☐ Minimum-volume threshold
- ☐ Statistical explanation visible

Backend

- ☐ API errors handled
- ☐ Model failures handled
- ☐ Database migrations reproducible

Frontend

- ☐ Empty states
- ☐ Loading states
- ☐ Error states
- ☐ Evidence visible
- ☐ Responsive enough for judging

Demo

- ☐ <3 minutes
 - ☐ Hidden cluster works
 - ☐ Backup mode works
 - ☐ Screenshots ready
 - ☐ Video recorded
-

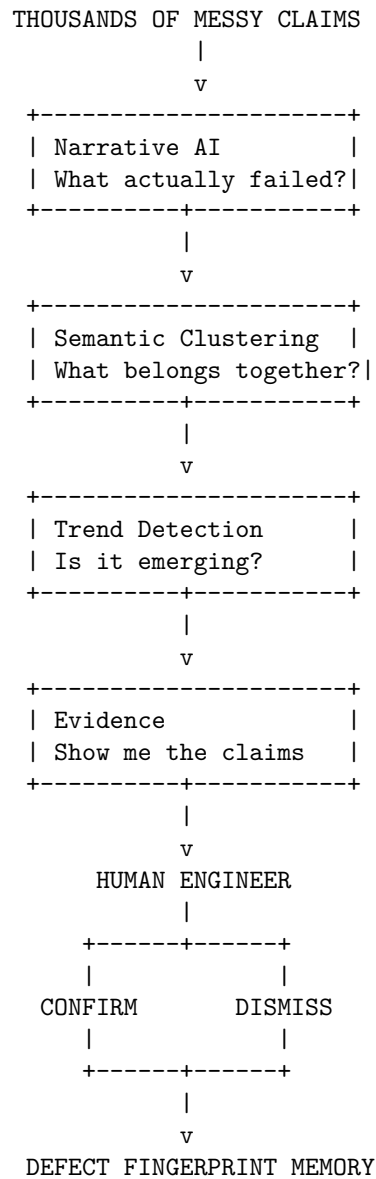
48. Definition of the MVP

The MVP is complete when a judge can watch you:

1. Load a messy warranty dataset.
2. See that structured failure-code analysis produces no meaningful alert.
3. Run WarrantyPatternMiner.
4. Watch the system discover a semantic cluster spanning multiple codes.
5. See that cluster's abnormal growth over time.
6. Open the exact source claims that support the finding.
7. Confirm the pattern.

8. See the system preserve that decision as defect knowledge.
- Everything else is secondary.

49. The Product in One Diagram



50. Final Positioning

Do not present WarrantyPatternMiner as:

“AI for warranty management.”

Present it as:

“An early-warning intelligence system for field defects that discovers the patterns a manufacturer’s own failure taxonomy hides.”

The product’s strongest conceptual insight is:

The defect is not necessarily hidden in the data. It is hidden by the way the data is categorized.

WarrantyPatternMiner uses AI to look underneath that categorization, discover semantic patterns, quantify whether those patterns are emerging, and provide the evidence a human engineer needs to investigate early.

51. Build Priority Matrix

Feature	Priority	Demo Value	Complexity
Dataset + ingestion	P0	6	2
Narrative extraction	P0	8	4
Embeddings	P0	8	3
Cross-code clustering	P0	10	5
Trend detection	P0	10	4
Evidence drill-down	P0	10	3
Structured-only baseline	P0	10	2
Feedback	P1	8	3
Fingerprint library	P1	8	3
8D draft	P2	5	3
Real-time streaming	P3	2	8
OEM integration	P3	2	9

P0 features must be finished before any P1/P2 work.

52. Final Build Rule

When time becomes limited, preserve this chain at all costs:

MESSY NARRATIVES

->

SEMANTIC HIDDEN CLUSTER

->

STATISTICALLY EMERGING TREND

->

SOURCE EVIDENCE

->

HUMAN CONFIRMATION

If that loop works flawlessly, you have the core product.

Do not sacrifice it to add more screens, more AI agents, or more integrations.